

Unit III - Searching and Sorting

Unit III - Searching and Sorting

- Searching is the process of finding a certain data item from a given list of values.
- The searching process is considered successful if the specified data item is successfully found during the searching process otherwise, if the specified data item is not found then the result is declared unsuccessful.
- The search operation comes to an end or is terminated when the specified item is found.
- This may also be possible that more than one instance of the searched item may be present in the provided list.
- A variety of search methods can be used (depending on the situation) for searching information. The most commonly used types of the searches methods are given below.
 1. **Sequential search and**
 2. **Binary search**

Linear Search

- Linear search is a very basic and simple search algorithm.
- In Linear search, we search an element or value in a given array by traversing the array from the starting, till the desired element or value is found.
- It compares the element to be searched with all the elements present in the array and when the element is matched successfully, it returns the index of the element in the array, else it return -1.
- Linear Search is applied on unsorted or unordered lists, when there are fewer elements in a list.

Features of Linear Search Algorithm

1. It is used for unsorted and unordered small list of elements.
2. It has a time complexity of $O(n)$, which means the time is linearly dependent on the number of elements, which is not bad, but not that good too.
3. It has a very simple implementation.

Implementing Linear Search

Following are the steps of implementation that we will be following:

1. Traverse the array using a for loop.
2. In every iteration, compare the target value with the current value of the array.
3. If the values match, return the current index of the array.
4. If the values do not match, move on to the next array element.
5. If no match is found, return -1.

Linear search algorithm

Linear Search(list, key)

for each item in the list

if item == value

return its index position

return -1

Python Program

```
def linear_Search(list1, n, key):

    # Searching list1 sequentially
    for i in range(0, n):
        if (list1[i] == key):
            return i
    return -1

list1 = [1 ,3, 5, 4, 7, 9]
key = 7

n = len(list1)
res = linear_Search(list1, n, key)
if(res == -1):
    print("Element not found")
else:
    print("Element found at index: ", res)
```

C++ Program

```
int SeqSearch (int A[max], int key, int n)
{
    int i, flag = 0, position;
    for(i = 0; i < n; i++)
    {
        if(key == A[i])
        {
            position = i;
            flag = 1;
            break;
        }
    }
    if(flag == 1)    // if found return position
        return(position);
    else    // return -1 if not found
        return(-1);
}
```

Computational Complexity

- In general, for any search algorithm, the computational complexity is computed by considering the number of comparisons made.
- The number of comparisons depends on where the target data is stored in the search list.
- If the target data is placed at the first location, we get it in just one comparison. Two comparisons are needed if the target data is in the second location.
- Similarly, i comparisons are required if the target data is at the i^{th} location and n comparisons, if it is at the n^{th} location.
- As the total number of comparisons depends on the position of the target data, let us compute the average complexity of the algorithm.
- **Average complexity** is the sum of number of comparisons for each position of the target data divided by n and is given as follows:
- **Average number of comparisons** = $(1 + 2 + 3 + \dots + n)/n$
$$= (S_n)/n$$
$$= ((n(n + 1))/2) \times 1/n$$
$$= (n + 1)/2$$

- Hence, the average number of comparisons done by the sequential search method in the case of a successful search is $(n + 1)/2$.
- An unsuccessful search is given by n comparisons.
- The number of comparisons is n and the complexity is denoted as **$O(n)$** .
- The **worst case complexity** is **n** , which means that the target data element is at the n th location and hence requires n comparisons.
- The **best case complexity** is **1** , as the target data element is at the first location and requires only a single comparison.

Pros

1. A simple and easy method
2. Efficient for small lists
3. Suitable for unsorted data
4. Suitable for storage structures which do not support direct access to data, for example, magnetic tape, linked list, etc.
5. Best case is one comparison, worst case is n comparisons, and average case is $(n + 1)/2$ comparisons
6. Time complexity is in the order of n denoted as $O(n)$.

Cons

1. Highly inefficient for large data
2. In the case of ordered data other search techniques such as binary search are found more suitable.

Variations of Sequential Search - Sentinel search

- The algorithm can be modified to eliminate the end of list test by placing the target at the end of list as just one additional entry.
- This additional entry at the end of the list is called as a ***sentinel***.
- Now, we need not test for the end of list condition within the loop and merely check after the loop completes whether we found the actual target or the sentinel.
- This modification avoids one comparison within the loop that varies n times.
- The only care to be taken is not to consider the sentinel entry as a data member

Algorithm

1. Set $i = 0$
2. $\text{key}[n] = \text{target}$ (add sentinel)
3. Compare $\text{key}[i]$ and target
if($\text{key}[i] = \text{target}$)
 Set $\text{location} = i$ and goto step 6
4. Move to next data element
 $i = i + 1$
5. goto step 3
6. if($\text{location} < n$) then
 return location as position of target
7. else
 report as 'Target not found' and return -1
8. stop

Implementing Sentinel Search in Python

```
def sentinel(lst, target):  
    size = len(lst)  
    lst.append(target)  
    i = 0  
    while(lst[i] != target):  
        i += 1  
    if(i == size):  
        return None  
    else:  
        return i
```

Implementing Sentinel Search in C++

```
int SeqSearch_sentinel (int A[max], int key, int n)
{
    int i, position;
    A[n] = key; // place target at end of the list
    while(key != A[i])
    {
        i = i + 1;
    }
    //if found at sentinel then return position
    if(i < n)
        return(i);
    else // return -1 if not found
        return(-1);
}
```

Binary Search

- This method is called binary search, as we have divided the list to be searched every time into two lists and the search is done in only one of the lists.
- Consider that the list is sorted in ascending order.
- In binary search algorithm, to search for a particular element, it is first compared with the element at the middle position, and if it is found, the search is successful, else if the middle position value is greater than the target, the search will continue in the first half of the list; otherwise, the target will be searched in the second half of the list.
- The same process is repeated for one of the halves of the list till the list is reduced to size one.

Binary search is implemented using following steps...

Step 1 - Read the search element from the user.

Step 2 - Find the middle element in the sorted list.

Step 3 - Compare the search element with the middle element in the sorted list.

Step 4 - If both are matched, then display "Given element is found!!!" and terminate the function.

Step 5 - If both are not matched, then check whether the search element is smaller or larger than the middle element.

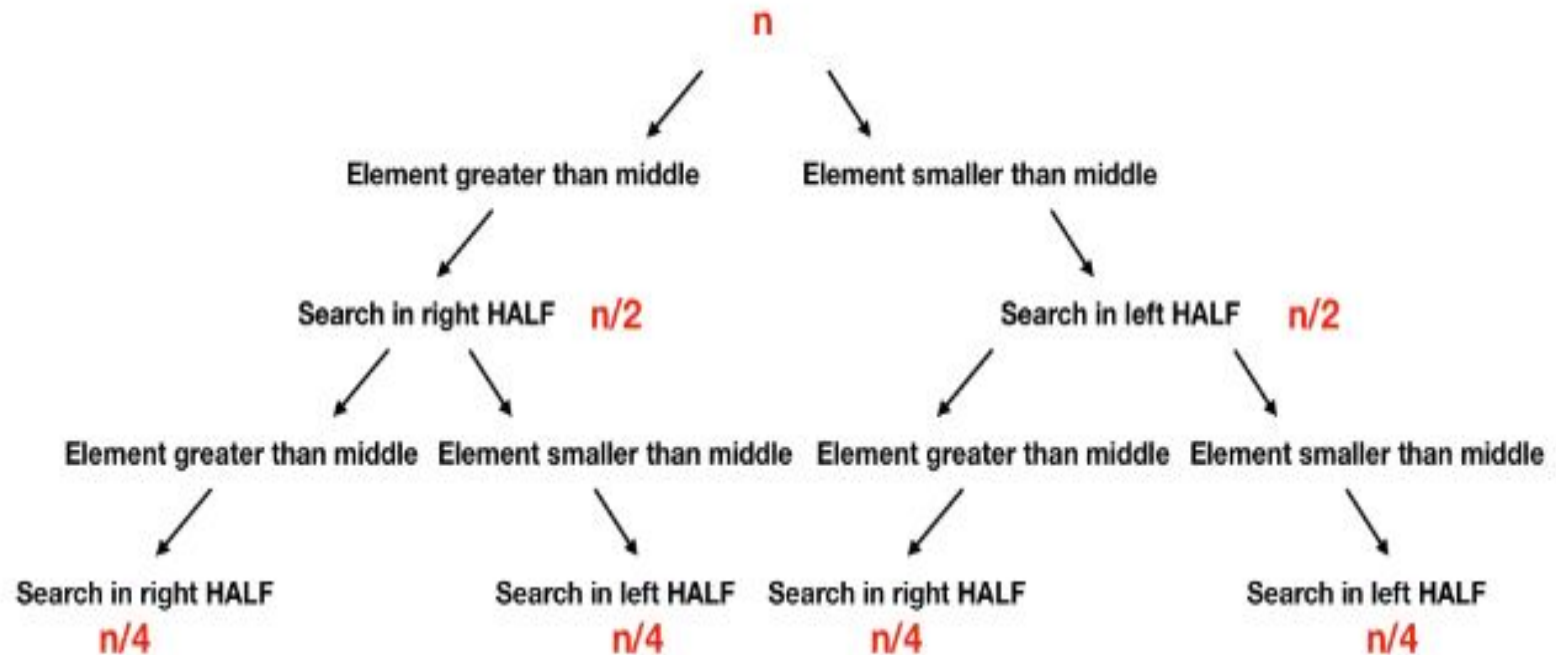
Step 6 - If the search element is smaller than middle element, repeat steps 2, 3, 4 and 5 for the left sublist of the middle element.

Step 7 - If the search element is larger than middle element, repeat steps 2, 3, 4 and 5 for the right sublist of the middle element.

Step 8 - Repeat the same process until we find the search element in the list or until sublist contains only one element.

Step 9 - If that element also doesn't match with the search element, then display "Element is not found in the list!!!" and terminate the function.

In binary search, without doing any analysis, we can see that the array is divided into half its initial size each time. So even in the worst case, it would end up searching only $\log_2 n$ elements.



Thus, binary search is a $O(\log_2 n)$ algorithm.

```
BINARY-SEARCH(A, start, end, x)
```

```
    if start <= end
```

```
        middle = floor((start+end)/2)
```

```
        if A[middle]==x
```

```
            return middle
```

```
        if A[middle]>x
```

```
            return BINARY-SEARCH(A, start, middle-1, x)
```

```
        if A[middle]<x
```

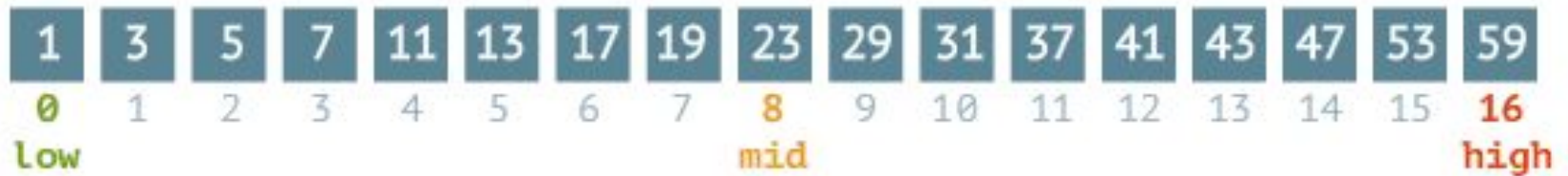
```
            return BINARY-SEARCH(A, middle+1, end, x)
```

```
    return FALSE // in case, element is not in the array
```

```
def binary_search(a, start, end, x):  
    if(start <= end):  
        middle = (start+end)//2  
        if(a[middle] == x):  
            return middle  
        if(a[middle] > x):  
            return binary_search(a, start, middle-1, x)  
        if(a[middle] < x):  
            return binary_search(a, middle+1, end, x)  
    return -1 # not found  
  
a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
index_result = binary_search(a, 0, 9, 7)  
print( index_result )
```

Binary search

steps: 0



Sequential search

steps: 0



Analysis of Binary Search

- In the base case, the algorithm will end up either finding the element or just failing and returning false.
- In both cases, the algorithm is going to take a constant time because only comparison and return statements are going to be executed.
- Otherwise, comparison and calculation of the middle element will take constant time and then the problem is divided into another problem of size $n/2$
- So, the overall running time ($T(n)$) can be written as:

$$T(n) = T\left(\frac{n}{2}\right) + c$$

Pros and Cons of Binary Search

The following are the pros and cons of a binary search:

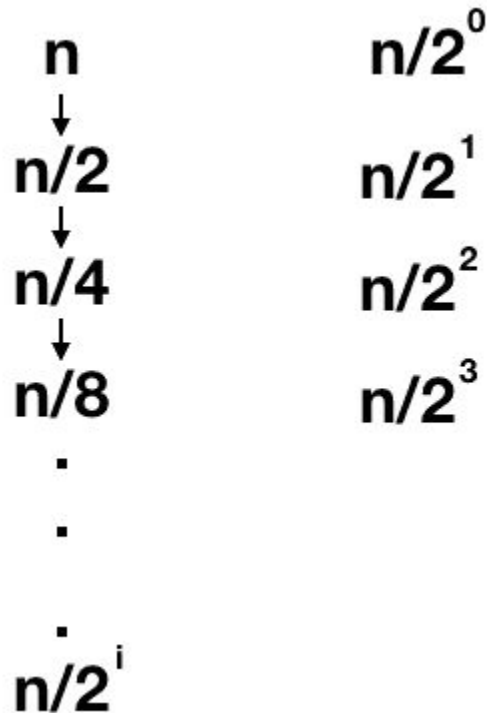
Pros

1. Suitable for sorted data
2. Efficient for large lists
3. Suitable for storage structures that support direct access to data
4. Time complexity is $O(\log_2(n))$

Cons

1. Not applicable for unsorted data
2. Not suitable for storage structures that do not support direct access to data, for example, magnetic tape and linked list
3. Inefficient for small lists

- Basically, we are reducing our problem to half the size in each iteration.



- In general, we can write the size of the problem as **$n/2^i$** after doing **i** comparisons.
- Thus, for the base case, **$n/2^i = 1 \Rightarrow i = \log_2 n$** .
- Also, the comparison will reach the base case only in the worst case and thus, binary search is a **$O(\log_2 n)$** algorithm.

Fibonacci Search

- Fibonacci search is a searching technique that depends on the Fibonacci numbers and it is based on the divide and conquer principle.
- The Fibonacci numbers are generated as:

$F(n + 1) = F(n) + F(n - 1)$ where $F(i)$ is the i th Fibonacci number.

- $F(0) = 0$ and $F(1) = 1$, these are two initial values of the Fibonacci series.
- First few Fibonacci numbers are: 0, 1, 1, 2, 3, 5, 8, 13, 21,

Fibonacci Search Algorithm

- **Step 1** – As the first step, find the immediate Fibonacci number that is greater than or equal to the size of the input array. Then, also hold the two preceding numbers of the selected Fibonacci number, that is, we hold F_m , F_{m-1} , F_{m-2} numbers from the Fibonacci Series.
- **Step 2** – Initialize the **offset** value as -1, as we are considering the entire array as the searching range in the beginning.

Step 3 – Until F_{m-2} is greater than 0, we perform the following steps –

1. Compare the key element to be found with the element at index **$[\min(\text{offset} + F_{m-2}, n-1)]$** . If a match is found, return the index.
2. If the key element is found to be lesser value than this element, we reduce the range of the input from 0 to the index of this element. The Fibonacci numbers are also updated with $F_m = F_{m-2}$.
3. But if the key element is greater than the element at this index, we remove the elements before this element from the search range. The Fibonacci numbers are updated as $F_m = F_{m-1}$. The offset value is set to the index of this element.

Step 4 – As there are two 1s in the Fibonacci series, there arises a case where your two preceding numbers will become 1. So if F_{m-1} becomes 1, there is only one element left in the array to be searched. We compare the key element with that element and return the 1st index. Otherwise, the algorithm returns an unsuccessful search.

EXAMPLE:

Suppose the lists of elements are:

50	58	64	74	80	100	120
0	1	2	3	4	5	6

Suppose the element is **100** which is to be searched.

The value of **n** is **7**

So, the smallest **Fibonacci number** which is equal to or greater than 7 is 8.

$F(m) = 8$, $F(m - 1) = 5$ and $F(m - 2) = 3$

First offset = -1

$i = \min(\text{offset} + 3, n - 1)$

$= \min(-1 + 3, 6)$

$= \min(2, 6)$

$= 2$

50	58	64	74	80	100	120
0	1	2	3	4	5	6

So, the **value of index 2 is 64** will be compared with the searched element (100).

As **64 is lesser than 100**

$F(m) = 5$, $F(m - 1) = 3$, $F(m - 2) = 2$ and the offset = 2

Therefore,

$i = \min(\text{offset} + 3, n - 1)$

$= \min(2 + 3, 6)$

$= \min(5, 6)$

$= 5$

50	58	64	74	80	100	120
0	1	2	3	4	5	6

Again, the value of index 5 is 100 which is the same as searched element 100. As the elements are matched the position of this element will be printed.

Pseudocode

Begin Fibonacci Search

n <- size of the input array

offset = -1

Fm2 := 0

Fm1 := 1

Fm := Fm2 + Fm1

while Fm < n do:

 Fm2 = Fm1

 Fm1 = Fm

 Fm = Fm2 + Fm1

done

while Fm > 1 do:

 i := minimum of (offset + fm2, n - 1)

 if (A[i] < x) then:

 Fm := Fm1

 Fm1 := Fm2

 Fm2 := Fm - Fm1

 offset = i

 end

else if (A[i] > x) then:

 Fm = Fm2

 Fm1 = Fm1 - Fm2

 Fm2 = Fm - Fm1

end

else

 return i;

end

done

if (Fm1 and Array[offset + 1] == x) then:

 return offset + 1

end

return invalid location;

end

```
def fibonacci_search(arr, n, key):
    offset = -1
    Fm2 = 0
    Fm1 = 1
    Fm = Fm2 + Fm1
    while (Fm < n):
        Fm2 = Fm1
        Fm1 = Fm
        Fm = Fm2 + Fm1
    while (Fm > 1):
        i = min(offset + Fm2, n - 1)
        if (arr[i] < key):
            Fm = Fm1
            Fm1 = Fm2
            Fm2 = Fm - Fm1
            offset = i
        elif (arr[i] > key):
            Fm = Fm2
            Fm1 = Fm1 - Fm2
            Fm2 = Fm - Fm1
        else:
            return i
    if (Fm1 == 1 and arr[offset + 1] == key):
        return offset + 1
    return -1
```

```
arr = [12, 14, 16, 17, 20, 24, 31, 43, 50, 62]
n = len(arr);
key = 20
index = fibonacci_search(arr, n, key)
if(index >= 0):
    print("The element is found at index: ",
(index))
else:
    print("Unsuccessful Search")
```


- **Analysis**

- The Fibonacci Search algorithm takes logarithmic time complexity to search for an element. Since it is based on a divide and conquer approach and is similar to idea of binary search, the time taken by this algorithm to be executed under the worst case consequences is $O(\log n)$.
- Fibonacci search is more efficient than binary search for large lists. However, it is inefficient in case of small lists.

- **Pros**

1. Faster than binary search for larger lists
2. Suitable for sorted lists

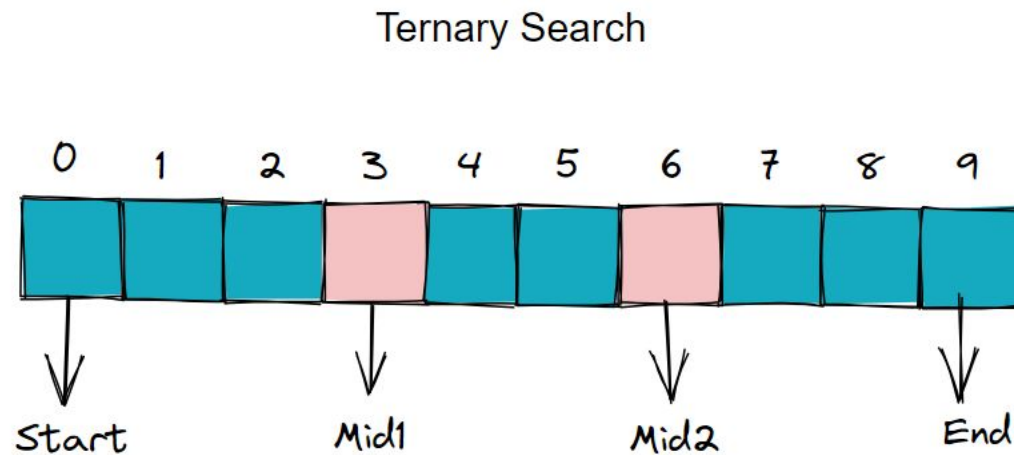
- **Con**

1. Inefficient for smaller lists

Ternary Search

- In binary search, the sorted array is divided into two parts while in ternary search, it is divided into 3 parts and then you determine in which part the element exists.
- Ternary search, like binary search, is a divide-and-conquer algorithm. It is mandatory for the array (in which you will search for an element) to be sorted before you begin the search. In this search, after each iteration it neglects $\frac{1}{3}$ part of the array and repeats the same operations on the remaining $\frac{2}{3}$.

- The ternary search works the same but divides the array into three equal parts, as its name implies. Ternary search works only for the sorted data. In this Answer, we'll see discuss how the ternary search works.



- The idea is to divide the array into three equal parts and check which part has the key element (the element to be found). It works the same as a binary search. The only difference is that it reduces the time complexity and divides the array into three parts rather than two.
- We can divide the array by taking two midpoints, mid1 and mid2. We can calculate midpoints as shown below:

$$\begin{aligned} mid1 &= start + \frac{(end - start)}{3} \\ mid2 &= end - \frac{(end - start)}{3} \end{aligned}$$

1. Initially, the value of start is zero and end is $n-1$, where n is the total number of elements in the array.
2. Let's see the steps we need to perform for the ternary search:
 1. Compare the key element with $mid1$. If it is found, return $mid1$.
 2. If not, compare the key element with $mid2$. If it is found, return $mid2$.
 3. Compare the key element with $mid1$. If it is less, then recur to the first part.
 4. Compare the key element with $mid2$. If it is greater, then recur to the third part. Otherwise, recur to the middle part.

Example

Value to search, key = 8

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----

Let's start ternary search for 8

Begin

Algorithm

if start $\leq$ end then

midFirst := start + (end - start) / 3

midSecond := midFirst + (end - start) / 3

if array[midFirst] = key then

return midFirst

if array[midSecond] = key then

return midSecond

if key < array[midFirst] then

call ternarySearch(array, start, midFirst-1, key)

if key > array[midSecond] then

call ternarySearch(array, midFirst+1, end, key)

else

call ternarySearch(array, midFirst+1, midSecond-1, key)

else

return invalid location

End

```
def ternary_search(arr, target):  
    left = 0  
    right = len(arr) - 1  
    while left <= right:  
        # Calculate two midpoints  
        mid1 = left + (right - left) // 3  
        mid2 = right - (right - left) // 3  
        if arr[mid1] == target:  
            return mid1  
        elif arr[mid2] == target:  
            return mid2  
        if target < arr[mid1]:  
            right = mid1 - 1  
        elif target > arr[mid2]:  
            left = mid2 + 1  
        else:  
            left = mid1 + 1  
            right = mid2 - 1  
    return -1 # Target element not found in the list
```



```
# Example usage:
arr = [1, 3, 5, 7, 9, 11, 13, 15, 17]
target = 9

index = ternary_search(arr, target)

if index != -1:
    print("Element {target} found at index {index}")
else:
    print("Element {target} not found in the list")
```

Indexed Sequential Search

- Indexed sequential search is suitable for sequential files.
- A sequential file with an associated index is just like an index associated with books.
- File index is a data structure similar to a list of keys and their location or reference to the location of the record associated with the key.
- An index file can be used to effectively overcome the problem associated with sequential files and to speed up the key search.

Table 9.1 Data file

Record position	Emp. no.	Name	Occupation
1	100	Saurabh	Developer
2	500	Aboleee	Project head
3	300	Shweta	Developer
4	200	Vaishali	Project head
5	400	Santosh	Developer

Its corresponding index file is given in Table 9.2.

Table 9.2 Index file of Table 9.1

Emp. no. (Key)	Record position
100	1
200	4
300	3
400	5
500	2

- Searching a record from this index file involves the following issues:
 1. The index file is ordered, so the searching can be done using the binary search method.
 2. The search is successful if we find the target element in the index.
 3. The record position is used to access the details of that record from the data file.

Algorithm for the Index Sequential Search :

Define a function **index_sequential_search(arr, key, block_size)** that takes the input list **arr**, the **key** you want to search for, and the **block_size** as parameters.

1. Create an empty list called **index** to store the starting indices of each block.
2. Iterate through the **arr** to create the **index**:
 1. Initialize a variable **i** to 0.
 2. While **i** is less than the length of **arr**:
 3. Append the current value of **i** to the **index** list.
 4. Increment **i** by **block_size**.

4. Initialize a variable **block_number** to -1. This variable will store the block where the key may be located.
5. Iterate through the **index** list:
 - If the value at **arr[index[i]]** is less than or equal to the key, update the **block_number** to the current index **i**.
6. If **block_number** is still -1, return -1, indicating that the key is not present in the array.
7. Perform a linear search within the identified block:
 - Iterate through the elements of the block, starting from **index[block_number]** up to the minimum of **index[block_number] + block_size** and the length of **arr**.
 - If an element in this range matches the key, return its index.
8. If the key is not found within the block, return -1 to indicate that the key is not in the array.

```
def index_sequential_search(arr, key, block_size):
    n = len(arr)
    index = []

    # Create an index for the blocks
    for i in range(0, n, block_size):
        index.append(i)

    # Find the block where the key may be located
    block_number = -1
    for i in range(len(index)):
        if arr[index[i]] <= key:
            block_number = i

    # If the block_number is still -1, the key is not present in the
    array
    if block_number == -1:
        return -1

    # Perform linear search within the identified block
    for i in range(index[block_number], min(index[block_number] +
block_size, n)):
        if arr[i] == key:
            return i
    return -1 # Key not found
```

Sorting

- **Sorting** is the operation of arranging the records of a table according to the key value of each record, or it can be defined as the process of converting an unordered set of elements to an ordered set.

Types of Sorting

- Sorting algorithms are divided into two categories: **internal and external sorts**.
- If all the records to be sorted are kept internally in the main memory, they can be sorted using an internal sort.
- However, if there are a large number of records to be sorted, they must be kept in external files on auxiliary storage. They have to be sorted using external sort.

Internal Sorting

- Any sort algorithm that uses main memory exclusively during the sorting is called as an internal sort algorithm. This assumes high-speed and random access to all data members.
- Internal sorting is faster than external sorting. The various internal sorting techniques are the following:
 1. Bubble sort
 2. Insertion sort
 3. Selection sort
 4. Quick sort
 5. Heap sort
 6. Shell sort
 7. Bucket sort
 8. Radix sort
 9. File sort
 10. Merge sort

External Sorting

- Any sort algorithm that uses external memory, such as tape or disk, during the sorting is called as an external sort algorithm. Merge sort uses external memory.

General Sort Concepts

1. Sort Order -

- Data can be ordered either in ascending or in descending order.
The order in which the data is organized, either ascending or descending, is called sort order.

2. Sort Stability -

- A sorting method is said to be stable if at the end of the method, identical elements occur in the same relative order as in the original unsorted set.
- A sorting algorithm is said to be stable if it preserves the order for all records with duplicate keys .
- Bubble sort, selection sort, and insertion sort are the stable sort methods.

Example

Name	Uma	Saurabh	Sanika	Kasturi	Ashish	Harsha	Lelo
Marks	80	90	93	95	83	90	83

Solution The stable sort method will sort the sequence as

Name	Kasturi	Sanika	Saurabh	Harsha	Ashish	Lelo	Uma
Marks	95	93	90	90	83	83	80

whereas, the unstable sort method may sort the same sequence as

Name	Kasturi	Sanika	Harsha	Saurabh	Lelo	Ashish	Uma
Marks	95	93	90	90	83	83	80

3. **Sort Efficiency**

- **Sort efficiency** is a measure of the relative efficiency of a sort. It is usually an estimate of the number of comparisons and data movement required to sort.

4. ***Passes***

- During the sorted process, the data is traversed many times. Each traversal of the data is referred to as a sort pass.
- Depending on the algorithm, the sort pass may traverse the whole list or just a section of the list.

Bubble Sort

- The bubble sort is the oldest and the simplest sort in use.
- The bubble sort works by comparing each item in the list with the item next to it and swapping them if required.
- The algorithm repeats this process until it makes a pass all the way through the list without swapping any items.
- This causes larger values to 'bubble' to the end of the list while smaller values 'sink' towards the beginning of the list.

Algorithm

```
1. Let A be the array to be sorted
2. for i = 1 to n - 1
    for j = 0 to n - i
    begin
        if A[j] > A[j+1] then
            Swap A[j] with A[j + 1] as follows
            temp = A[j]
            A[j] = A[j + 1]
            A[j + 1] = temp
        end
    end
end
3. stop
```

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(0, n-i-1):  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
    return arr
```

Test the implementation with an example list

```
arr = [64, 34, 25, 12, 22, 11, 90]  
print("Before sorting: ", arr)  
arr = bubble_sort(arr)  
print("After sorting: ", arr)
```


6 5 3 1 8 7 2 4

Analysis of Bubble Sort

The time complexity for each of the cases is given by the following:

1. Average case complexity = $O(n^2)$
2. Best case complexity = $O(n^2)$
3. Worst case complexity = $O(n^2)$

Insertion Sort

- The insertion sort works just like its name suggests—it inserts each item into its proper place in the final list.
- The simplest implementation of this requires two list structures: the source list and the list into which the sorted items are inserted.
- Let us consider a list $L = \{3, 6, 9, 14\}$. Given this sorted list, we need to insert a new element 5 in it. The commonly used process would involve the following steps:
 1. Compare the new element 5 and the last element 14
 2. Shift 14 right to get 3, 6, 9, ,14
 3. Shift 9 right to get 3, 6, ,9, 14
 4. Shift 6 right to get 3, ,6, 9, 14
 5. Insert 5 to get 3, 5, 6, 9, 14

6 5 3 1 8 7 2 4

Pseudo Code for Insertion Sort

insertion_sort(A):

 for $i = 1$ to $n-1$:

$key = A[i]$

$j = i - 1$

 while $j \geq 0$ and $A[j] > key$:

$A[j+1] = A[j]$

$j = j - 1$

$A[j+1] = key$

```
def insertion_sort(alist):
    for i in range(1, len(alist)):
        temp = alist[i]
        j = i - 1
        while (j >= 0 and temp < alist[j]):
            alist[j + 1] = alist[j]
            j = j - 1
        alist[j + 1] = temp

alist = input('Enter the list of numbers: ').split()
alist = [int(x) for x in alist]
insertion_sort(alist)
print('Sorted list: ', end='')
print(alist)
```

Analysis of Insertion Sort

- Time and Space Complexity of Insertion Sort in Python
- Since Insertion sort uses nested for-loops it is a slow algorithm that can be extensively slow in the case of larger arrays but still for smaller-size arrays insertion sort can be a great choice.
- **Complexity of the Insertion Sort Algorithm**
- To sort an unsorted list with 'n' number of elements, we need to make $(1+2+3+\dots+n-1) = (n(n-1))/2$ number of comparisons in the worst case. If the list is already sorted then it requires 'n' number of comparisons.
- **Worst Case : $O(n^2)$**
- **Best Case : $\Omega(n)$**
- **Average Case : $\Theta(n^2)$**

Selection Sort

- The selection sort algorithms construct the sorted sequence, one element at a time, by adding elements to the sorted sequence in order. At each step, the next element to be added to the sorted sequence is selected from the remaining elements.
- Both selection and insertion sorts sort the arrays in-place
- In this method, we sort a set of unsorted elements in two steps.
- In the first step, find the smallest element in the structure.
- In the second step, swap the smallest element with the element at the first position.
- Then, find the next smallest element and swap with the element at the second position.
- Repeat these steps until all elements get arranged at proper positions

5 3 4 1 2

Selection Sort

Selection Sort Algorithm

```
def selection_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n-1):
```

```
        min_index = i
```

```
        for j in range(i+1, n):
```

```
            if arr[j] < arr[min_index]:
```

```
                min_index = j
```

```
    arr[i], arr[min_index] = arr[min_index], arr[i]
```

```
def selection_sort(alist):
    for i in range(0, len(alist) - 1):
        smallest = i
        for j in range(i + 1, len(alist)):
            if alist[j] < alist[smallest]:
                smallest = j
        alist[i], alist[smallest] = alist[smallest], alist[i]

alist = input('Enter the list of numbers: ').split()
alist = [int(x) for x in alist]
selection_sort(alist)
print('Sorted list: ', end='')
print(alist)
```

Quick Sort

- Quicksort is a divide-and-conquer algorithm.
- It works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot.
- For this reason, it is sometimes called partition-exchange sort.
- The sub-arrays are then sorted recursively.
- This can be done in-place, requiring small additional amounts of memory to perform the sorting.

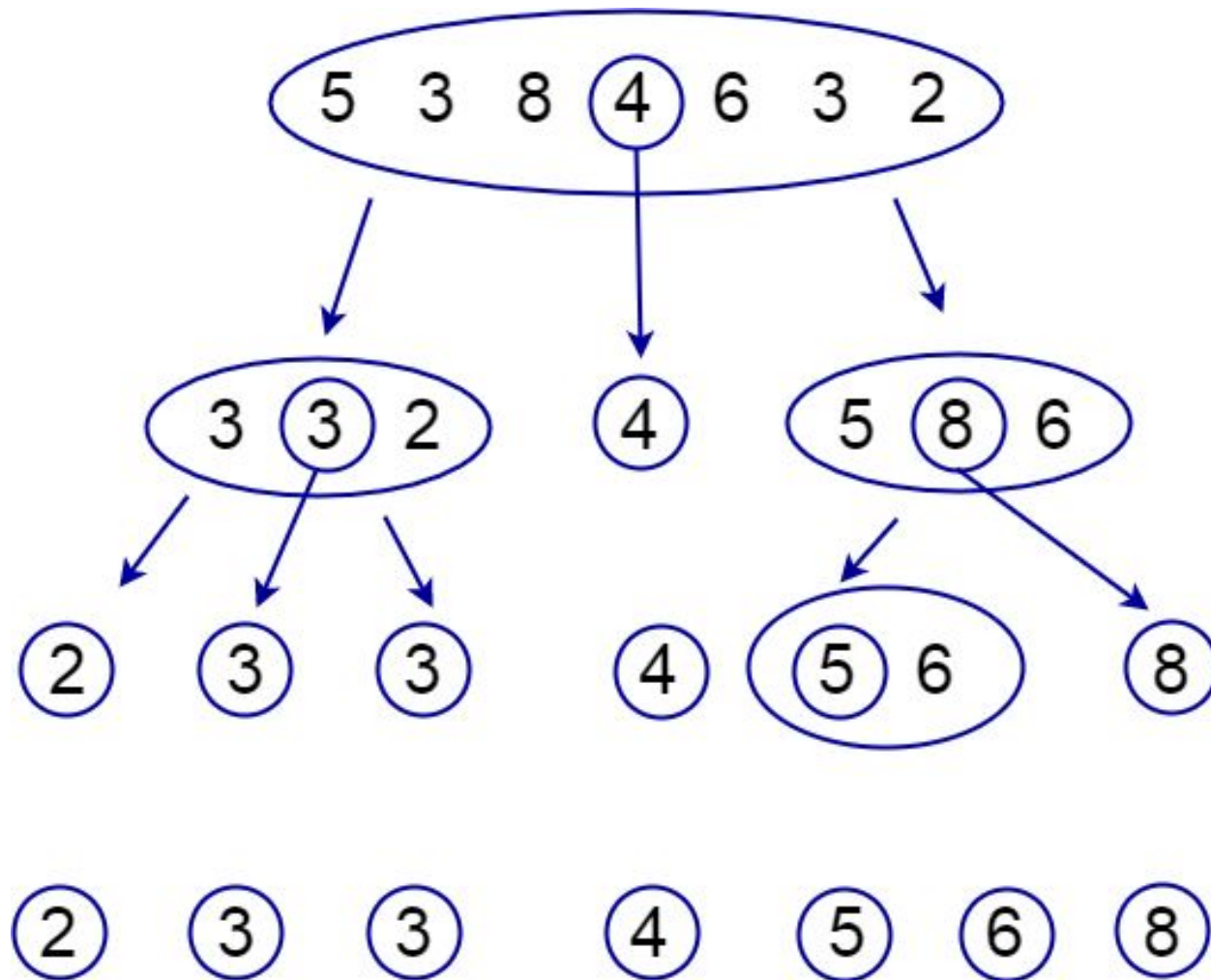
Quick Sort Algorithm

1. Select an element from the array as pivot.
2. Partition the array.
3. One partition will have all the elements that are smaller than the pivot.
4. Another partition will have other elements that are greater than the pivot.
5. Apply this procedure recursively with these two partitions.

We take different strategies based on various conditions to select the pivot to get better performance.

1. First element of the partitions is selected as pivot.
2. Last Element of the partitions is selected as pivot.
3. Random element of the partitions is selected as pivot.
4. Median of first, middle and last element of the partition is selected as pivot.
5. Graphical Illustration of Quick Sort

Graphical Illustration of Quick Sort



Let us break down the Quicksort process into a few steps.

1. Select a pivot.
2. Initialize the left and right pointers, pointing to the left and right ends of the list.
3. Start moving the left and right pointers toward the pivot while their values are smaller and greater than the pivot, respectively.
4. At each step, check and place the elements smaller than the pivot to the left of the pivot, and elements greater than that to the right.
5. When the two pointers meet or cross each other, we have completed one iteration of the list and the pivot is placed in its correct position in the final sorted array.
6. Now, two new lists are obtained on either side of the pivot.
7. Repeat steps 1–5 on each of these lists until all elements are placed in their correct positions.

```
def partition(A, start, end):  
    i = start #left pointer  
    pivot = A[(start+end)//2] # pivot  
    j = end #right pointer  
    while True:  
        while (A[i] < pivot):  
            i+=1 #move left pointer to right  
        while (A[j]> pivot):  
            j-=1 #move right pointer to left  
        if i>=j:  
            return j #stop, pivot moved to its correct position  
        A[i],A[j] = A[j], A[i]
```



```
def quickSort(A, start, end):  
    if start < end:  
        p = partition(A, start, end)  
        quickSort(A, start, p-1)  
        quickSort(A, p+1, end)  
  
A = [24, 10, 30, 13, 20, 27]  
  
print("Original array A:",A )  
  
quickSort(A, 0, len(A)-1)  
  
print("Array A after quicksort:",A)
```

Quicksort Time Complexity

- For an input of size n , it is divided into parts k and $n-k$ at each step.
- So, Time complexity for n elements = Time complexity for k elements + Time complexity for $n-k$ elements + Time complexity for selecting the pivot

$$T(n)=T(k)+T(n-k)+M(n)$$

Best Case

- The best-case complexity occurs when the middle element is selected as the pivot in each recursive loop.
- The array is divided into lists of equal size at each iteration and as this process is repeated, the sorting is completed in the minimum number of steps possible.
- The number of recursions performed will be $\log(n)$ with n operations at each step.
- Hence, the time complexity is obtained to be **$O(n(\log(n)))$** .

Worst Case

- In the worst-case scenario, n number of recursion operations are performed and the time complexity is **$O(n^2)$** .
- This can occur under the following conditions:
 1. The smallest or largest element is selected as a pivot at each step.
 2. The last element is selected as the pivot and the list is already in increasing or decreasing order.

Average Case

- The average case is obtained by considering an average of the time complexities of the various permutations of the array. The complexity is **$O(n \log(n))$** .

Shell Sort

- Shell Sort is an extended version of insertion sort.
- The fundamental idea of this sorting algorithm is to group the elements that are far apart and sort them accordingly.
- Then gradually decrease the gap between them.
- Shell sort overcomes the average case time complexity of insertion sort by comparing and exchanging elements that are far away.
- This gap, known as the interval, is reduced according to some optimal gap sequences.
- The running time of shell sort is also dependent on these sequences.
- There are several gap sequences, such as Shell's original sequence, Knuth's formula, Hibbard's increments, etc.
- Shell's original gap sequence is – $n/2, n/4, n/8, \dots, 1$

We use several sequences to reduce the interval between the elements to be sorted. We will use Shell's original sequence($N/2$, $N/4$,, 1) for the implementation of the Shell Sort Algorithm.

Index	0	1	2	3	4	5	6	7
Given Array	9	8	3	7	5	6	4	1
Interval $N/2 = 4$	9	8	3	7	5	6	4	1
Interval $N/2 = 2$	5	6	3	1	9	8	4	7
Interval $N/2 = 1$	3	1	4	6	5	7	9	8
Sorted Array	1	3	4	5	6	7	8	9

Algorithm:

Step 1 – Start

Step 2 – Initialize the value of gap size. Example: h

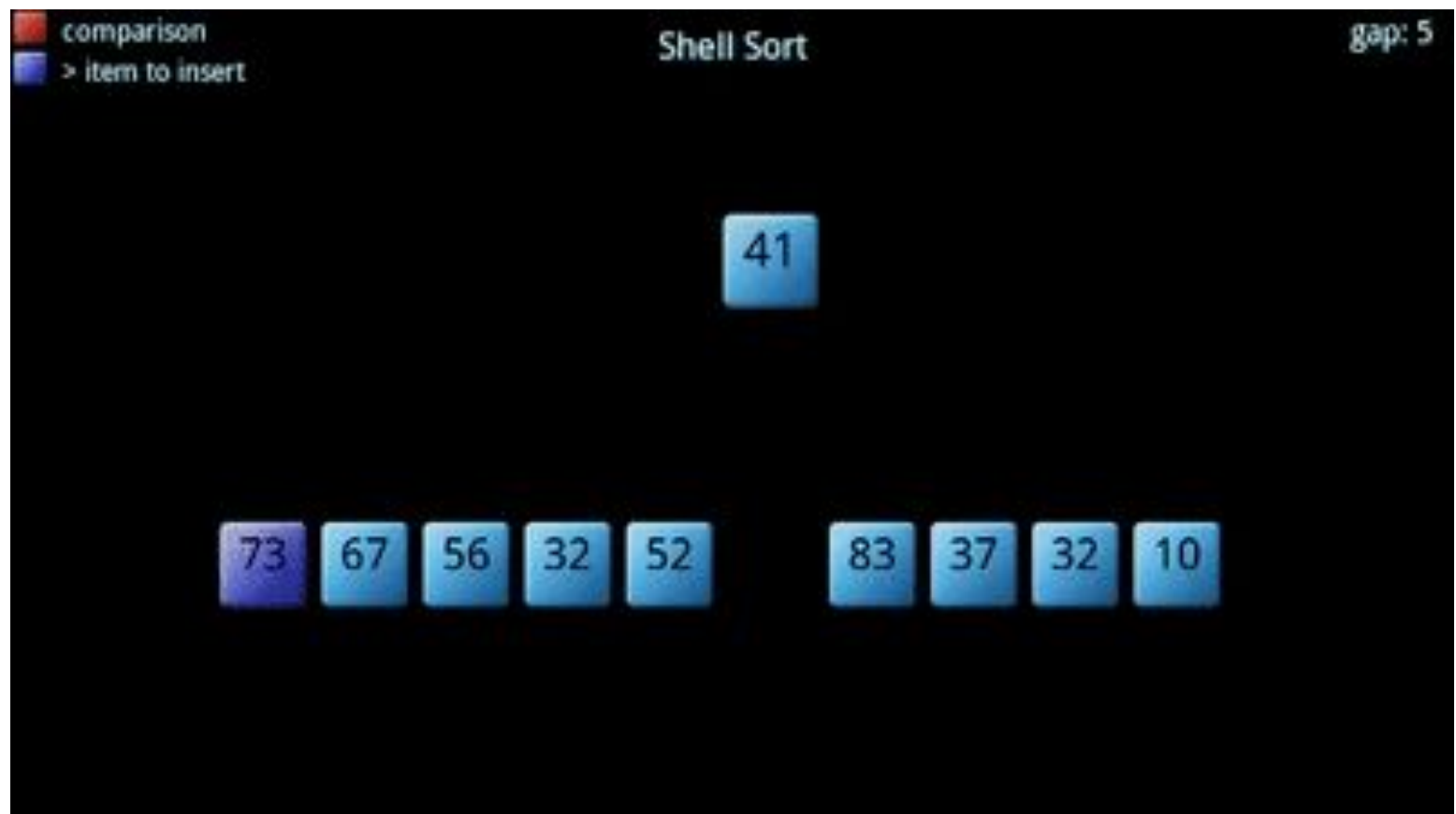
Step 3 – Divide the list into smaller sub-part. Each must have equal intervals to h

Step 4 – Sort these sub-lists using insertion sort

Step 5 – Repeat this step 2 until the list is sorted.

Step 6 – Print a sorted list.

Step 7 – Stop.



Shell Sort Algorithm

The steps or procedure for the shell sort algorithm is as follows-

1. Initialize the interval value, **interval** = $n/2$. (In this example, n is the size of the array)
2. Put all the elements within a distance of the interval in a sub-list.
3. Sort those sub-lists using insertion sort.
4. Set new interval, **interval** = **interval** / 2.
5. If **interval** > 0, go to step 2. Else go to step 6.
6. The resultant output will be the sorted array.

Shell sort in python

```
def shellSort(array, n):  
    # Rearrange elements at each n/2, n/4, n/8, ... intervals  
    interval = n // 2  
    while interval > 0:  
        for i in range(interval, n):  
            temp = array[i]  
            j = i  
            while j >= interval and array[j - interval] > temp:  
                array[j] = array[j - interval]  
                j -= interval  
            array[j] = temp  
        interval //= 2  
data = [9, 8, 3, 7, 5, 6, 4, 1]  
size = len(data)  
shellSort(data, size)  
print('Sorted Array in Ascending Order:')  
print(data)
```

Shell Sort Complexity

Time Complexity	
Best	$O(n \log n)$
Worst	$O(n^2)$
Average	$O(n \log n)$
Space Complexity	
$O(1)$	
Stability	
No	

Merge Sort

- Merge Sort is based on the Divide and Conquer Algorithm.
- It divides the input list into two halves, calls itself recursively for the two halves, and then merges the two sorted halves.
- Here, a problem is divided into multiple sub-problems.
- Each sub-problem is solved individually.
- Finally, sub-problems are combined to form the final solution.
- MergeSort has worst-case and average-case complexity of $O(n \cdot \log(n))$ which makes it fastest than some of the other sorting algorithms.

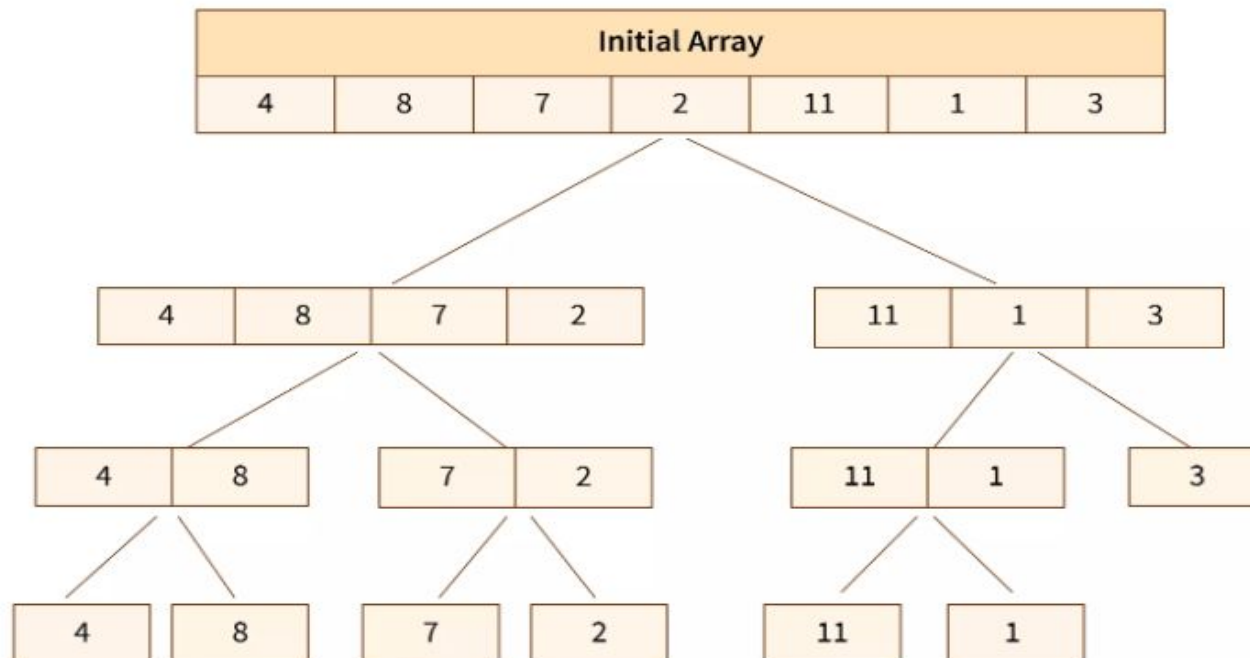
Let us consider an array = [4, 8, 7, 2, 11, 1, 3].

1. **Divide** - For dividing the array, we calculate the midpoint of the array by using the formula

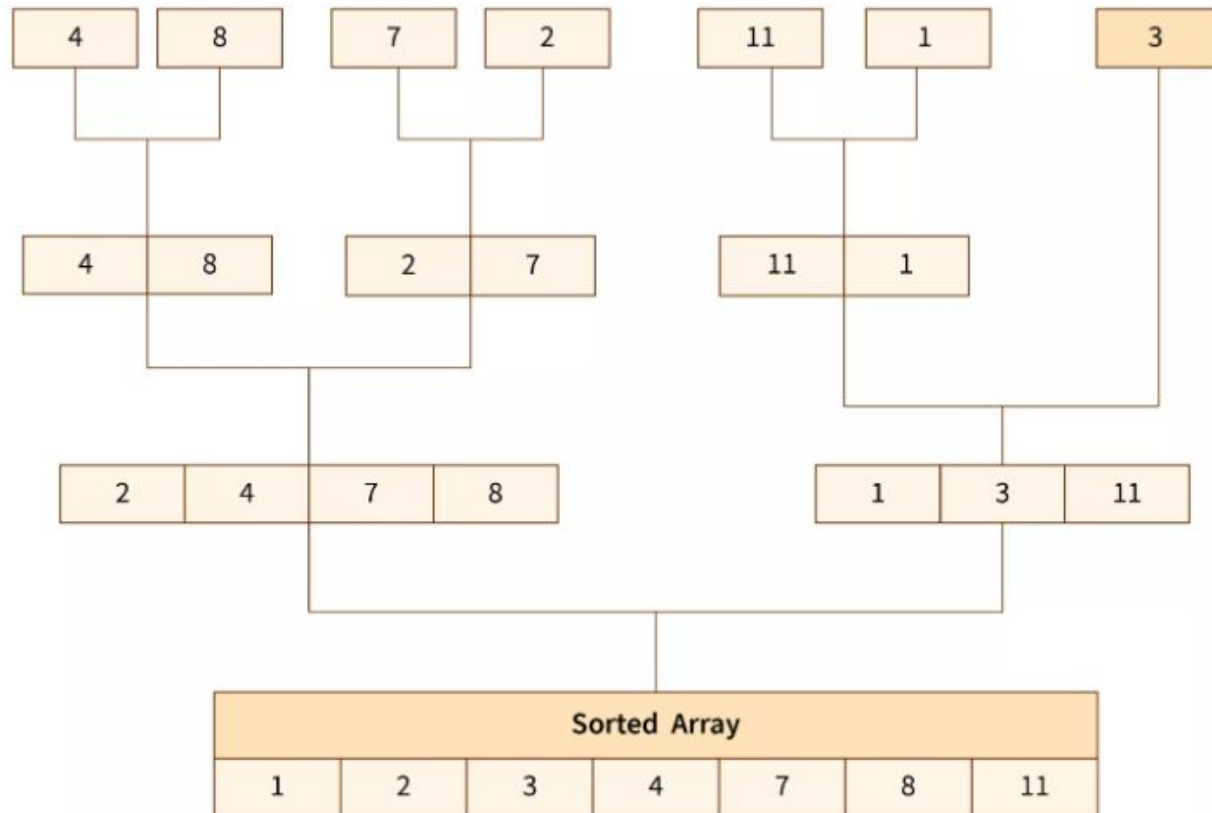
$$mid = len(array)//2$$

2. **Conquer** -In this step, we divide the array into sub-arrays with the help of the midpoint calculated.

This division into sub-arrays and calculation of midpoint is done recursively for the same. As we know, a single element in an array is always sorted. Therefore, we have to continuously divide the sub-arrays until all elements in the array are single array elements.



3. **Combine** - Since, now we are done with the formation of sub-arrays, the last step is to combine them back in sorted order.



MergeSort Algorithm

Input: List a of n elements.

Output: Returns a new list containing the same elements in sorted order.

Algorithm:

1. If less than two elements, return a copy of the list (***base case!***)
2. Sort the first half using merge sort. (***recursive!***)
3. Sort the second half using merge sort. (***recursive!***)
4. Merge the two sorted halves to obtain the final sorted array.

Merge Function Algorithm

- **Input:** Two lists a and b, already sorted
- **Output:** A new list containing the elements of a and b merged together in sorted order.
- **Algorithm:**
 1. Create an empty list c, set index_a and index_b to 0
 2. While index_a < length of a and index_b < length of b a. Add the smaller of a[index_a] and b[index_b] to the end of c b. increment the index of the list with the smaller element
 3. If any elements are left over in a or b, add them to the end of c, in order
 4. Return c

```
def msort(list):  
    if len(list) == 0 or len(list) == 1: # base case  
        return list[:len(list)] # copy the input  
  
    # recursive case  
  
    m = len(list) // 2  
  
    list1 = list[0: m]  
  
    list2 = list[m :len(list)]  
  
    newlist1 = msort(list1) # recursively sort left half  
  
    newlist2 = msort(list2) # recursively sort right half  
  
    newlist = merge(newlist1, newlist2)  
  
    return newlist
```



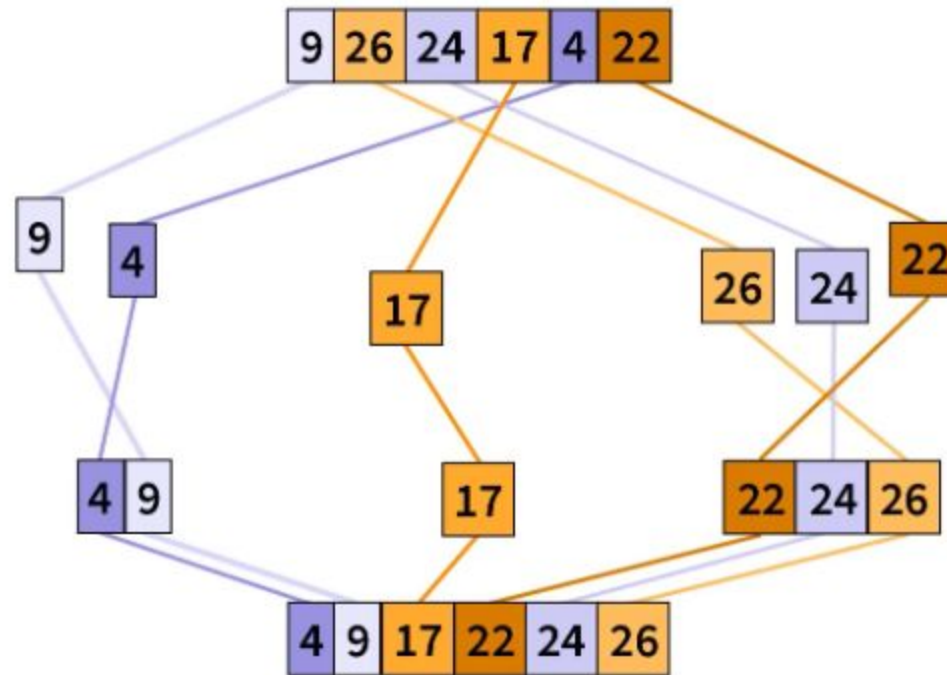
```
def merge(a, b):  
    index_a = 0  
    index_b = 0  
    c = []  
    while index_a < len(a) and index_b < len(b):  
        if a[index_a] <= b[index_b]:  
            c.append(a[index_a])  
            index_a = index_a + 1  
        else:  
            c.append(b[index_b])  
            index_b = index_b + 1  
  
    # when we exit the loop  
  
    # we are at the end of at least one of the lists  
    c.extend(a[index_a:]) # The extend() method adds the specified list  
c.extend(b[index_b:]) # elements to the end of the current list.  
    return c
```

Non-comparison Based Sorting Methods -Bucket Sort

- Bucket Sort is a sorting algorithm that divides the unsorted array elements into several groups called buckets.
- Each bucket is then sorted by using any of the suitable sorting algorithms or recursively applying the same bucket algorithm.
- It works on **uniformly distributed data** (explained below) and puts numbers into buckets to sort and gather them all to form the sorted array.
- **Uniformly Distributed Data**
 - Uniformly distributed data has difference between every adjacent element almost the same.
 - Examples and graph to represent uniformly distributed data.
 - For example:
 - Consider this array: **[10, 21, 29, 41, 52]** The difference between each adjacent term is almost equal to 10. Hence, this array has uniformly distributed data and can be sorted using the bucket sort algorithm.

Scatter-Gather-Approach

- Scatter-gather-approach tries to simplify such complex problems by first scattering the complete data into clusters, solving these sub-problems individually, and finally gathering them all together to get the final result.
- This is how Bucket sort uses scatter-gather-approach:



Bucket Sort Algorithm

- Works on Uniformly Distributed data
- Follows Scatter-Gather-Approach
- Sorting technique with time complexity $O(n)$
- It divides the unsorted array into separate groups and calls them buckets. Sort the individual buckets, and then gather them all together to form the final sorted array.
- If the elements of the array are ***floats*** ranging between 0 and 1, we primarily make 10 buckets, numbered from 0 to 9, and then insert elements into these buckets depending upon their most significant number. The bucket index is calculated as: $(\text{int})(\text{elementNumber} * 10)$
- If the elements of the array are ***integers***, as shown in the figure below, we simply calculate the range: $\text{range} = (\text{maximumNumber} - \text{minimumNumber}) / \text{noOfBuckets}$ and divide the whole range into buckets and then perform bucket sorting.

11	9	21	8	17	19	13	1	24	12
----	---	----	---	----	----	----	---	----	----

↓ Scatter



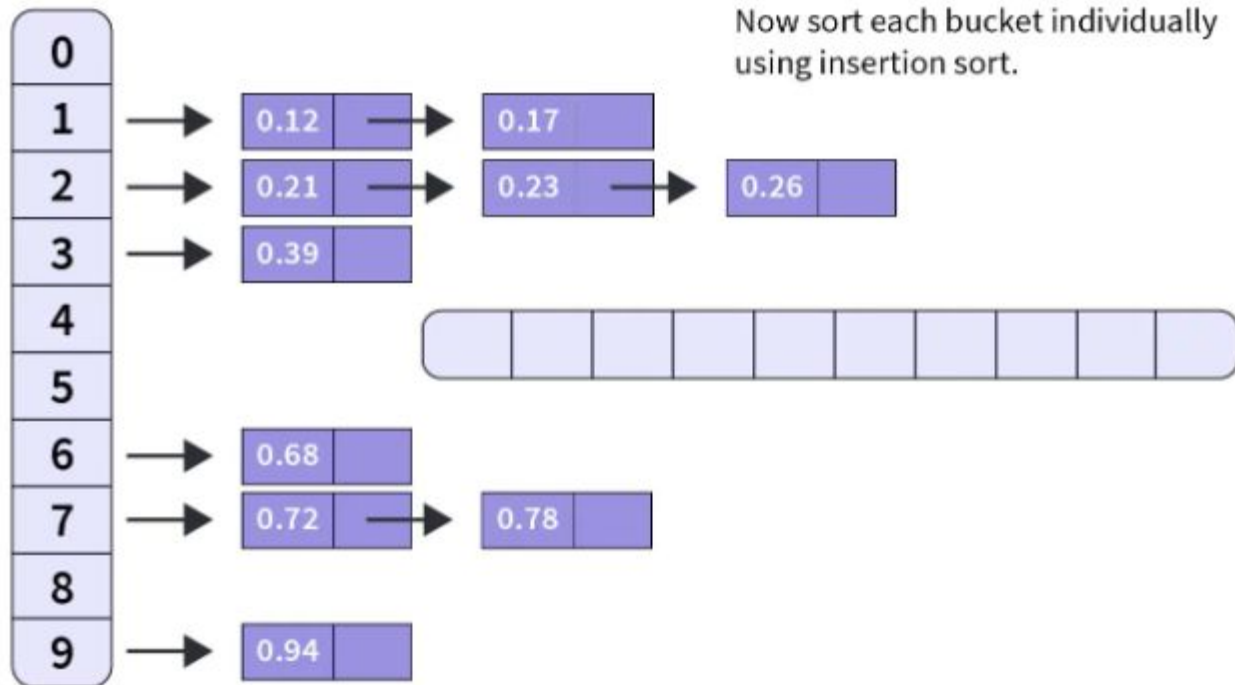
↓ Sort



↓ Gather

1	8	9	11	12	13	17	19	21	24
---	---	---	----	----	----	----	----	----	----

0.78	0.17	0.39	0.26	0.72	0.94	0.21	0.12	0.23	0.68
------	------	------	------	------	------	------	------	------	------



Bucket Sort Algorithm

bucketSort()

create N buckets each of which can hold a range of values

for all the buckets

initialize each bucket with 0 values

for all the buckets

put elements into buckets matching the range

for all the buckets

sort elements in each bucket

gather elements from each bucket

end bucketSort

Bucket Sorting for Integer elements

1. Find range = $(\text{max} - \text{min}) / \text{noOfBuckets}$
2. $\text{bucketIndex} = (\text{array}[i] - \text{minimum}) / \text{range}$
3. Finally, sort and gather
4. For integer elements, we need to follow the following algorithm:
 1. Calculate the maximum and the minimum element of the array
 2. Calculate the range: $\text{range} = (\text{maximum} - \text{minimum}) / n$, where n is the number of buckets (Given as parameter)
 3. Create n empty buckets and initialize them with 0
 4. Loop through the unsorted array and perform the following:
 1. Calculate bucketIndex $\text{bucketIndex} = (\text{array}[i] - \text{minimum}) / \text{range}$
 2. Insert the i th element of the array into the $\text{bucket}[\text{bucketIndex}]$
 5. Sort the individual buckets
 6. Gather all the elements together


```
def bucket_sort(arr):
```

```
    if len(arr) == 0:
```

```
        return arr
```

```
    # Find the minimum and maximum values in the input array
```

```
    min_val, max_val = min(arr), max(arr)
```

```
    # Create empty buckets
```

```
    num_buckets = max_val - min_val + 1
```

```
    buckets = [[] for _ in range(num_buckets)]
```

```
    # Place elements into buckets
```

```
    for num in arr:
```

```
        buckets[num - min_val].append(num)
```

```
    # Sort each bucket using a different sorting algorithm (e.g., insertion sort)
```

```
    for i in range(num_buckets):
```

```
        buckets[i].sort()
```

Concatenate the sorted buckets to get the final sorted array

```
sorted_arr = []
```

```
for bucket in buckets:
```

```
    sorted_arr.extend(bucket)
```

```
return sorted_arr
```

Example usage:

```
arr = [3, 6, 8, 1, 4, 9, 2, 5, 7]
```

```
sorted_arr = bucket_sort(arr)
```

```
print(sorted_arr)
```

Bucket Sort time complexity

Best Case Time Complexity: $O(n+k)$

Average Case Time Complexity: $O(n)$

Worst Case Time Complexity: $O(n^2)$

Best Case Time Complexity:

- If the array elements are uniformly distributed, bucket size will almost be the same for all the buckets. Hence, this will be the best case which will take up the least amount of time.
- To create n buckets and scatter each element from the array, time complexity = $O(n)$. If we use Insertion sort to sort each bucket, time complexity = $O(k)$. **Hence, best case time complexity for bucket sort = $O(n+k)$** , where n = number of elements, and k = number of buckets

Worst Case Time Complexity

- If the array elements are not uniformly distributed, i.e., elements are concentrated within specific ranges.
- This will result in one or more buckets having more elements than other buckets, making bucket sort like any other sorting technique, where every element is compared to the other.
- Time complexity increases even further if the elements in the array are present in the reverse order. If insertion sort is used, the worst-case time complexity can go up to **$O(n^2)$** .

Bucket Sort Space Complexity

- Space Complexity : **$O(n+k)$**
- Space Complexity for bucket sort is $O(n+k)$, where n = number of elements in the array, and k = number of buckets formed. Space taken by each bucket is $O(k)$, and inside each bucket, we have n elements scattered. Hence, the space complexity becomes $O(n+k)$.

Non-comparison Based Sorting Methods-Counting Sort

- Counting sort is a sorting algorithm that sorts an array's items by counting the number of times each unique element appears in the array.
- The count is saved in an auxiliary array, and sorting is accomplished by mapping the count to an auxiliary array index.
- It operates by determining the number of items with unique key values

Counting Sort Pseudo-code

1. Iterate the input array and find the maximum value present in it.
2. Declare a new array of size $\text{max}+1$ with value 0
3. Count each and every element in the array and increment its value at the corresponding index in the auxiliary array created
4. Find cumulative sum in the auxiliary array by adding curr and prev frequency
5. Now the cumulative value actually signifies the actual location of the element in the sorted input array
6. Start iterating auxiliary array from 0 to max
7. Put 0 at the corresponding index and reduce the count by 1, which will signify the second position of the element if it exists in the input array
8. Now transfer array received in the above step in the actual input array

Counting Sort Algorithm

countingSort(arr, n)

maximum <- find the largest element in arr

create a count array of size maximum+1

initialise count array with all 0's

for i <- 0 to n

find the total frequency/ occurrences of each element and

store the count at ith index in count arr

for i <- 1 to maximum

find the cumulative sum by adding current(i) and prev(i-1) count and store

it in count arr itself

for j <- n down to 1

copy the element back into the input array

decrement count of each element copied by 1

```
def countingSort(inputArray):  
    # Find the maximum element in the inputArray  
    maxElement= max(inputArray)  
    countArrayLength = maxElement+1  
    # Initialize the countArray with (max+1) zeros  
    countArray = [0] * countArrayLength  
    # Step 1 -> Traverse the inputArray and increase  
    # the corresponding count for every element by 1  
    for el in inputArray:  
        countArray[el] += 1  
    # Step 2 -> For each element in the countArray,  
    # sum up its value with the value of the previous  
    # element, and then store that value  
    # as the value of the current element  
    for i in range(1, countArrayLength):  
        countArray[i] += countArray[i-1]
```


Step 3 -> Calculate element position

based on the countArray values

```
outputArray = [0] * len(inputArray)
```

```
i = len(inputArray) - 1
```

```
while i >= 0:
```

```
    currentEl = inputArray[i]
```

```
    countArray[currentEl] -= 1
```

```
    newPosition = countArray[currentEl]
```

```
    outputArray[newPosition] = currentEl
```

```
    i -= 1
```

```
return outputArray
```

```
inputArray = [2,2,0,6,1,9,9,7]
```

```
print("Input array = ", inputArray)
```

```
sortedArray = countingSort(inputArray)
```

```
print("Counting sort result = ", sortedArray)
```

The Complexity of the Counting Sort Algorithm

- The Counting sort algorithm uses only simple for and while loops without any complex recursions and subroutine calls, therefore, its complexity analysis is a pretty straightforward process.
- Before we dive into the complexity analysis, let's label the length of the input array as n and the value of the maximum element in the input array as k .
- **Time Complexity**
 - The first step of the algorithm iterates over the input array n times in order to initialize the count array, so it has the complexity of **$O(n)$** .
 - The second step iterates over the count k times in order to calculate the cumulative sum of each element, so it has the complexity of **$O(k)$** .
 - The third step performs the sorting based on the counting array, so it has to iterate in a while loop n times, therefore it has the complexity of **$O(n)$** .
 - Collectively, the time complexity of the Counting Sort algorithm is **$O(n+k)$** .

Space Complexity

- Counting sort uses input and output array, both of length n and one count array of length $(k+1)$.
- Therefore, the total space that this algorithm uses is **$O(n+k)$** .

Non-comparison Based Sorting Methods - Radix Sort

- Radix sort is one of the sorting algorithms used to sort a list of integer numbers in order.
- In radix sort algorithm, a list of integer numbers will be sorted based on the digits of individual numbers.
- Sorting is performed from least significant digit to the most significant digit.
- Radix sort algorithm requires the number of passes which are equal to the number of digits present in the largest number among the list of numbers.
- For example, if the largest number is a 3 digit number then that list is sorted with 3 passes.

Step by Step Process

The Radix sort algorithm is performed using the following steps...

1. Define 10 queues each representing a bucket for each digit from 0 to 9.
2. Consider the least significant digit of each number in the list which is to be sorted.
3. Insert each number into their respective queue based on the least significant digit.
4. Group all the numbers from queue 0 to queue 9 in the order they have inserted into their respective queues.
5. Repeat from step 3 based on the next least significant digit.
6. Repeat from step 2 until all the numbers are grouped based on the most significant digit.

Example

Consider the following list of unsorted integer numbers

82, 901, 100, 12, 150, 77, 55 & 23

Step 1 - Define 10 queues each represents a bucket for digits from 0 to 9.



Step 2 - Insert all the numbers of the list into respective queue based on the Least significant digit (once placed digit) of every number.

82, 901, 100, 12, 150, 77, 55 & 23



Group all the numbers from queue-0 to queue-9 in the order they have inserted & consider the list for next step as input list.

100, 150, 901, 82, 12, 23, 55 & 77

Step 3 - Insert all the numbers of the list into respective queue based on the next Least significant digit (Tens placed digit) of every number.

100, 150, 901, 82, 12, 23, 55 & 77



Group all the numbers from queue-0 to queue-9 in the order they have inserted & consider the list for next step as input list.

100, 901, 12, 23, 150, 55, 77 & 82

Step 4 - Insert all the numbers of the list into respective queue based on the next Least significant digit (Hundred placed digit) of every number.

100, 901, 12, 23, 150, 55, 77 & 82



Group all the numbers from queue-0 to queue-9 in the order they have inserted & consider the list for next step as input list.

12, 23, 55, 77, 82, 100, 150, 901

List got sorted in the increasing order.

Pseudocode of the Radix Sort

- In pseudocode, we will be having the function RadixSort which is the main function implementing the Radix sort functionality.
- Firstly, the maximum element of the array is determined, then countingSort function is called k times where k is the number of digits in the maximum element found in the previous step.

```
RadixSort(a[], n):  
    // Finding the maximum element  
    max=a[0]  
    For (i=1 to n-1):  
        If (a[i]>max):  
            max=a[i]  
    // Calling countingSort for k times using For loop.  
    For (div=1 to max/div>0):  
        countingSort(a, n, div)  
        div=div*10
```



```
def countingSortForRadix(inputArray, placeValue):  
    # We can assume that the number of digits used to represent  
    # all numbers on the placeValue position is not greater than 10  
    countArray = [0] * 10  
    inputSize = len(inputArray)  
  
    # placeElement is the value of the current place value  
    # of the current element, e.g. if the current element is  
    # 123, and the place value is 10, the placeElement is  
    # equal to 2  
    for i in range(inputSize):  
        placeElement = (inputArray[i] // placeValue) % 10  
        countArray[placeElement] += 1  
    for i in range(1, 10):  
        countArray[i] += countArray[i-1]
```

Reconstructing the output array

```
outputArray = [0] * inputSize
i = inputSize - 1
while i >= 0:
    currentEl = inputArray[i]
    placeElement = (inputArray[i] // placeValue) % 10
    countArray[placeElement] -= 1
    newPosition = countArray[placeElement]
    outputArray[newPosition] = currentEl
    i -= 1

return outputArray
```

```
def radixSort(inputArray):  
    # Step 1 -> Find the maximum element in the input array  
    maxEl = max(inputArray)  
  
    # Step 2 -> Find the number of digits in the `max` element  
    D = 1  
    while maxEl > 0:  
        maxEl /= 10  
        D += 1  
  
    # Step 3 -> Initialize the place value to the least significant place  
    placeVal = 1  
  
    # Step 4  
    outputArray = inputArray  
    while D > 0:  
        outputArray = countingSortForRadix(outputArray, placeVal)  
        placeVal *= 10  
        D -= 1  
  
    return outputArray  
  
input = [2,20,61,997,1,619]  
print(input)  
sorted = radixSort(input)  
print(sorted)
```

Complexity of the Radix Sort Algorithm

To sort an unsorted list with 'n' number of elements, Radix sort algorithm needs the following complexities...

Worst Case : $O(n)$

Best Case : $O(n)$

Average Case : $O(n)$

Comparison of All Sorting Methods and their complexities

Sorting method	Technique in brief	Best case	Worst case	Memory requirement	Is stable?	Pros	Cons
Bubble sort	Repeatedly stepping through the list to be sorted, comparing each pair of adjacent items and swapping them if they are in the wrong order	$O(n^2)$	$O(n^2)$	No extra space needed	Yes	1. A simple and easy method 2. Efficient for small lists $n > 100$	Highly inefficient for large data
Selection sort	Finds the minimum value in the list and then swaps it with the value in the first position, repeats these steps for the remainder of the list (starting at the second position and advancing each time)	$O(n^2)$	$O(n^2)$	No extra space needed	No	1. Recommended for small files 2. Good for partially sorted data	Inefficient for large lists
Insertion sort	Every repetition of insertion sort removes an element from the input data, inserts it into the correct position in the already sorted list until no input elements remain. The choice of which element to remove from the input is arbitrary and can be made using almost any choice of algorithm	$O(n)$	$O(n^2)$	No extra space needed	Yes	1. Relatively simple and easy to implement 2. Good for almost sorted data	Inefficient for large lists

Sorting method	Technique in brief	Best case	Worst case	Memory requirement	Is stable?	Pros	Cons
Quick sort	Picks an element, called a pivot, from the list. Reorders the list so that all elements with values less than the pivot come before the pivot, whereas all elements with values greater than the pivot come after it (equal values can go either way). After this partitioning, the pivot is in its final position. This is called the partition operation. Recursively sorts the sub-list of the lesser elements and the sub-list of the greater elements.	$O(n \log_2 n)$	$O(n^2)$	No extra space needed	No	1. Extremely fast 2. Inherently recursive	Very complex algorithm
Shell sort	It is a generalization of insertion sort, which exploits the fact that insertion sort works efficiently on input that is already almost sorted. It improves on insertion sort by allowing the comparison and exchange of elements that are far apart. The last step of shell sort is a plain insertion sort, but by then, the array of data is guaranteed to be almost sorted	$O(n^{1.5})$	$O(n \log^2 n)$	No extra space needed	No	1. It is faster than a quick sort for small arrays 2. Its speed and simplicity makes it a good choice in practice	Slower for sufficiently big arrays

Sorting method	Technique in brief	Best case	Worst case	Memory requirement	Is stable?	Pros	Cons
Radix sort (most significant digit)	Numbers are placed at proper locations by processing individual digits and by comparing individual digits that share the same significant position.	$O(n)$	$O(n)$	Extra space proportional to n is needed	Yes	1. Radix sort is very simple and fast 2. In-Place, recursive, and one of the fastest sorting algorithms for numbers or strings of letters	Radix sort can also take more space than other sorting algorithms since in addition to the array that will be sorted, there needs to be a sub-list for each of the possible digits or letters
Merge sort	If the list is of length 0 or 1, then it is already sorted. Otherwise, the algorithm divides the unsorted list into two sub-lists of about half the size Then, it sorts each sub-list recursively by reapplying the merge sort and then merges the two sub-lists back into one sorted list.	$O(n \log_2 n)$	$O(n \log_2 n)$	Extra space proportional to n is needed	Yes	1. Good for external file sorting 2. Can be applied to files of any size	1. It requires twice the memory of the heap sort because of the second array used to store the sorted list. 2. It is recursive, which can make it a bad choice for applications that run on machines with limited memory
Heap sort	Heap sort begins by building a heap out of the data set, and then removing the largest item and placing it at the end of the partially sorted array. After removing the largest item, it reconstructs the heap, removes the largest remaining item, and places it in the next open position from the end of the partially sorted array. This is repeated until there are no items left in the heap and the sorted array is full	$O(n \log_2 n)$	$O(n \log_2 n)$	No extra space needed	No	1. Advantageous as it does not use recursion and that heap sort works just as fast for any data order. That is, there is basically no worst case scenario 2. Heaps work well for small tables and the tables where changes are infrequent	Do not work well for most large tables

Sorting Algorithms	Special Input Condition	Time Complexity			Space Complexity
		Best Case	Average Case	Worst Case	Worst Case
Counting Sort	Each input element is an integer in the range 0- K	$\Omega(N + K)$	$\Theta(N + K)$	$O(N + K)$	$O(K)$
Radix Sort	Given n digit number in which each digit can take on up to K possible values	$\Omega(NK)$	$\Theta(NK)$	$O(NK)$	$O(N + K)$
Bucket Sort	Input is generated by the random process that distributes elements uniformly and independently over the interval [0, 1)	$\Omega(N + K)$	$\Theta(N + K)$	$O(N^2)$	$O(N)$